

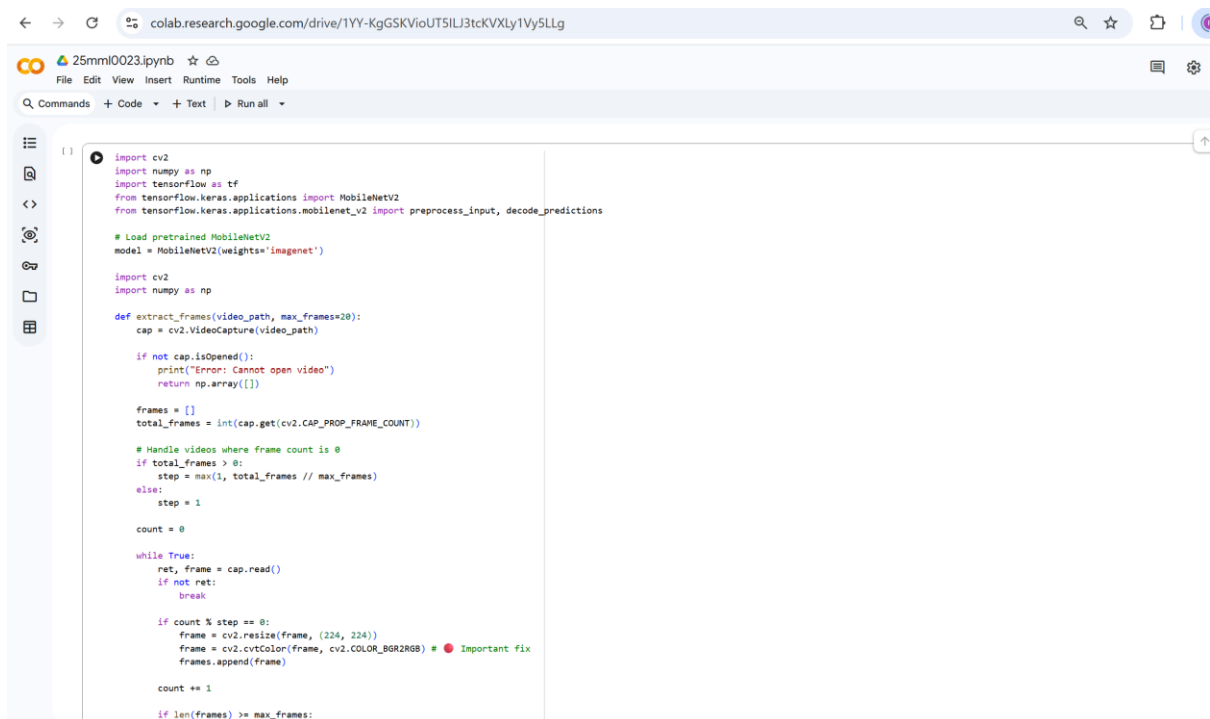
EDGE INTELLIGENCE

LAB-7

PRADHEESH RAJ R

25MML0023

CODE-1:



```
colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg

25mml0023.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text | ▶ Run all

import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# Load pretrained MobileNetV2
model = MobileNetV2(weights='imagenet')

import cv2
import numpy as np

def extract_frames(video_path, max_frames=30):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Handle videos where frame count is 0
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

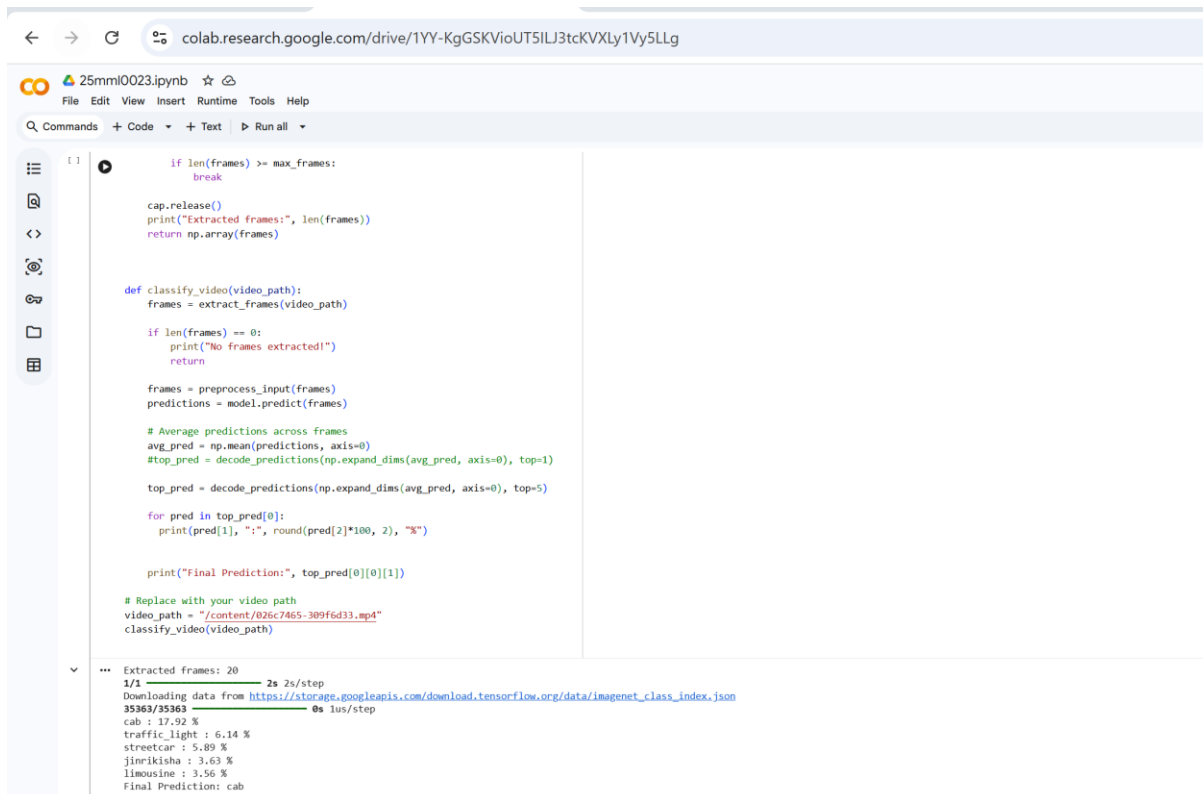
    count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Important fix
            frames.append(frame)

        count += 1

    if len(frames) >= max_frames:
```



```
11 if len(frames) >= max_frames:
12     break
13
14 cap.release()
15 print("Extracted frames:", len(frames))
16 return np.array(frames)
17
18 def classify_video(video_path):
19     frames = extract_frames(video_path)
20
21     if len(frames) == 0:
22         print("No frames extracted!")
23         return
24
25     frames = preprocess_input(frames)
26     predictions = model.predict(frames)
27
28     # Average predictions across frames
29     avg_pred = np.mean(predictions, axis=0)
30     #top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=1)
31     top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)
32
33     for pred in top_pred[0]:
34         print(pred[1], ":", round(pred[2]*100, 2), "%")
35
36     print("Final Prediction:", top_pred[0][0][1])
37
38 # Replace with your video path
39 video_path = "/content/026c7465-309f6d33.mp4"
40 classify_video(video_path)
```

... Extracted frames: 20
1/1 ————— 2s 2s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ————— 0s 1us/step
cab : 17.92 %
traffic_light : 6.14 %
streetcar : 5.89 %
jinrikisha : 3.63 %
limousine : 3.56 %
Final Prediction: cab

STEPS IN THE CODE:

1. Import required libraries
Import cv2, numpy, tensorflow, and MobileNetV2 related functions.
2. Load Pretrained Model
Load MobileNetV2 with weights='imagenet' for image classification.
3. Open Video File
Use cv2.VideoCapture(video_path) to read the video.
4. Check Video Open Status
Use cap.isOpened() to ensure the video loads correctly.
5. Calculate Frame Step Size
Compute $\text{step} = \text{total_frames} // \text{max_frames}$ to sample frames evenly.
6. Read Frames Sequentially
Use cap.read() inside a loop to extract frames.
7. Resize Frames
Resize each selected frame to (224, 224) as required by MobileNetV2.
8. Convert Color Format
Convert BGR $\rightarrow$ RGB using cv2.cvtColor().

9. Preprocess Frames

Apply `preprocess_input()` before feeding into the model.

10. Predict & Average Results

Predict each frame → Take mean of predictions → Decode top 5 classes
→ Print final prediction.

DRAWBACKS OF THIS CODE:

1. Uses ImageNet Model for Video
MobileNetV2 is trained for image classification, not video classification.
2. No Temporal Information
Frame averaging ignores motion and sequence information.
3. Simple Frame Sampling
May miss important action frames.
4. No Face/Object Detection Preprocessing
Entire frame is classified, not specific objects.
5. Limited to 20 Frames
Important details might be lost.
6. Not Suitable for Deepfake Detection
ImageNet classes cannot detect fake videos.
7. High Memory Usage for Large Videos
8. No Confidence Threshold Check
9. No GPU Optimization Handling
10. No Error Logging Mechanism

ERRORS IN CODE:

1. Hardcoded Video Path
2. `video_path = "/content/026c7465-309f6d33.mp4"`
Should be dynamic input.
3. No Batch Prediction Handling
For long videos, memory can crash.

4. No Exception Handling
Use try-except for safer execution.
5. No Model Summary Printed
Helpful for debugging.
6. No FPS-Based Frame Sampling
Better to sample based on time instead of frame count.
7. decode_predictions Used Without Class Mapping Validation
8. No Softmax Check
Ensure predictions are properly normalized.
9. No Logging for Debugging
10. Not Designed for Custom Dataset
Only works with ImageNet classes.

BETTER APPROACH (Video Classification):

Instead of MobileNetV2, use:

- 3D CNN (Conv3D)
- LSTM + CNN
- I3D (Inflated 3D ConvNet)
- TimeSformer
- SlowFast Network


```
colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C

25mmI0023.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

import cv2
import numpy as np
import torch
import matplotlib.pyplot as plt
from transformers import VideoMAEImageProcessor, VideoMAEForVideoClassification

# 1. Load Pretrained VideoMAE
model_ckpt = "McG-NJU/vidomae-base-finetuned-kinetics"
processor = VideoMAEImageProcessor.from_pretrained(model_ckpt)
model = VideoMAEForVideoClassification.from_pretrained(model_ckpt)

def extract_20_frames(video_path):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened(): return None

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    # Step 1: Extract 20 frames evenly from the video
    indices = np.linspace(0, total_frames - 1, 20, dtype=int)

    frames = []
    for i in range(total_frames):
        ret, frame = cap.read()
        if not ret: break
        if i in indices:
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)
        if len(frames) == 20: break
    cap.release()
    return frames

def display_frames(frames):
    """Plots all 20 extracted frames in a 5x4 grid."""
    fig, axes = plt.subplots(5, 4, figsize=(12, 15))
    for i, ax in enumerate(axes.flat):
        if i < len(frames):
            ax.imshow(frames[i])
            ax.set_title(f"Frame {i+1}")
            ax.axis('off')
        else:
            ax.axis('off')
    plt.tight_layout()
    plt.show()

def classify_with_vidomae(video_path):
    # Get your 20 frames
    video_frames_20 = extract_20_frames(video_path)
    if not video_frames_20 or len(video_frames_20) < 20:
        print("Error: Could not extract 20 frames.")
        return

    # Visualise the 20 frames
    display_frames(video_frames_20)
```

```
colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C

25mmI0023.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

# Visualise the 20 frames
display_frames(video_frames_20)

# Step 2: Select 16 frames from the 20 to satisfy the model's requirements
# This takes 16 evenly spaced frames from your 20-frame list
selection_indices = np.linspace(0, len(video_frames_20) - 1, 16, dtype=int)
video_frames_16 = [video_frames_20[i] for i in selection_indices]

# Step 3: Preprocess and Predict
inputs = processor(list(video_frames_16), return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)
    logits = outputs.logits

probs = torch.nn.functional.softmax(logits, dim=-1)
top_results = torch.topk(probs, 5)

print(f"\n--- Top 5 Predictions (Using 20-frame context) ---")
for i in range(5):
    score = top_results.values[0][i].item()
    label_id = top_results.indices[0][i].item()
    label = model.config.id2label[label_id]
    print(f"{label}: {round(score * 100, 2)}%")

# Usage
video_path = "/content/026c7465-309f6d33.mp4"
classify_with_vidomae(video_path)
```

Loading weights: 100% 186/186 [00:00<00:00, 563.74kB/s, Materializing param=vidomae.encoder.layer.11.output.dense.weight]

Frame 1

Frame 2

Frame 3

Frame 4



colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXly1Vy5LLg#scrollTo=PmqygtRoi6C

25mmI0023.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Usage
video_path = "/content/026c7465-389f6d33.mp4"
classify_with_video_mae(video_path)
```

Frame 5 Frame 6 Frame 7 Frame 8

Frame 9 Frame 10 Frame 11 Frame 12

Frame 13 Frame 14 Frame 15 Frame 16

colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXly1Vy5LLg#scrollTo=PmqygtRoi6C

25mmI0023.ipynb

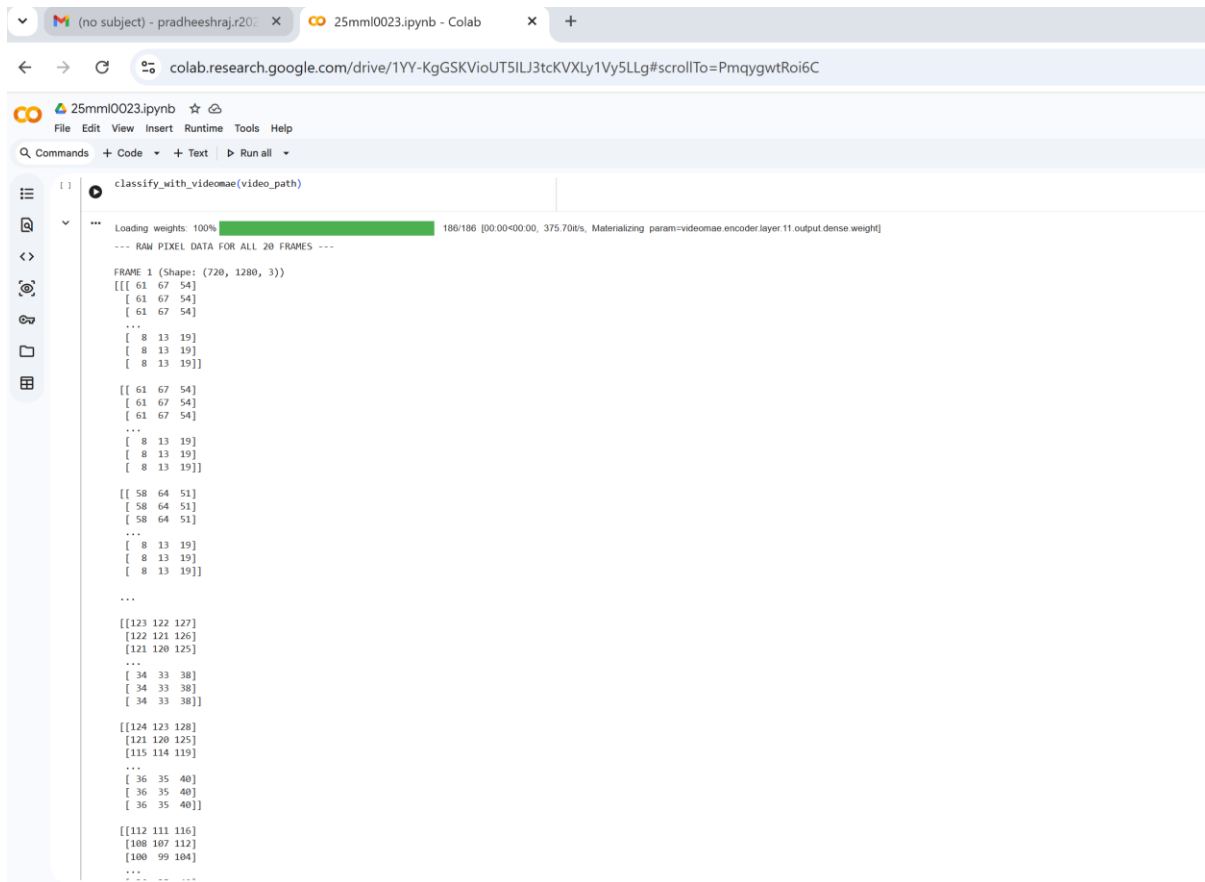
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Frame 17 Frame 18 Frame 19 Frame 20

Frame 21 Frame 22 Frame 23 Frame 24

```
--- Top 5 Predictions (Using 20-frame context) ---
pushing car: 18.87%
driving car: 17.09%
pushing cart: 5.15%
motorcycling: 3.08%
pushing wheelchair: 2.47%
```



```
classify_with_videomae(video_path)

Loading weights: 100% 196/196 [00:00<00:00, 375.70it/s, Materializing param=videomae.encoder.layer.11 output.dense.weight]
--- RAW PIXEL DATA FOR ALL 20 FRAMES ---

FRAME 1 (Shape: (720, 1280, 3))
[[[ 61  67  54]
 [ 61  67  54]
 [ 61  67  54]
 ...
 [ 8  13  19]
 [ 8  13  19]
 [ 8  13  19]]

 [[ 61  67  54]
 [ 61  67  54]
 [ 61  67  54]
 ...
 [ 8  13  19]
 [ 8  13  19]
 [ 8  13  19]]

 [[ 58  64  51]
 [ 58  64  51]
 [ 58  64  51]
 ...
 [ 8  13  19]
 [ 8  13  19]
 [ 8  13  19]]

 ...

 [[123 122 127]
 [122 121 126]
 [121 120 125]
 ...
 [ 34  33  38]
 [ 34  33  38]
 [ 34  33  38]]

 [[124 123 128]
 [121 120 125]
 [115 114 119]
 ...
 [ 36  35  40]
 [ 36  35  40]
 [ 36  35  40]]

 [[112 111 116]
 [108 107 112]
 [100  99 104]
 ...
 ...
 ...]]
```

CODE-2:

STEPS:

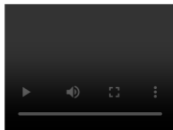
1. Import required libraries (cv2, numpy, matplotlib, subprocess, etc.).
2. Install OpenCV using pip install opencv-python (if not installed).
3. Import subprocess (optional, used for running system commands).
4. Provide the video file path.
5. Load the video using cv2.VideoCapture().
6. Check if the video opened successfully.
7. Read video metadata using cap.get() function.
8. Count total number of frames using cv2.CAP_PROP_FRAME_COUNT.
9. Get video frame width using cv2.CAP_PROP_FRAME_WIDTH.
10. Get video frame height using cv2.CAP_PROP_FRAME_HEIGHT.
11. Get frames per second (FPS) using cv2.CAP_PROP_FPS.

12. Read the first frame using `cap.read()`.
13. Understand that OpenCV loads images in BGR format.
14. Convert BGR image to RGB using `cv2.cvtColor()` for matplotlib display.
15. Use modulo operator (`frame_number % 100 == 0`) to extract every 100th frame.
16. Display frames using `matplotlib.pyplot.imshow()`.
17. Release video capture using `cap.release()`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm
```

```
#import subprocess
```

```
ipd.Video("026c7465-309f6d33.mp4", width = 200)
```



Start coding or generate with AI.

Open the Video and Read Metadata

```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

```
# Load in video capture
import cv2
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
```

```
# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

```
2398.0
```

```
# Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

```
Height 720.0, Width 1280.0
```

```
# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')
```

FPS : 59.94

```
cap.release()
```

Pull the image from the first frame

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read() #will return TRUE everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

Returned True and img of shape (720, 1280, 3)

Loading the First Image

```
plt.imshow(img) #opencv loads color in BGR format, matplotlib lib expects RGB format
```

<matplotlib.image.AxesImage at 0x7faad2e85be0>



```
## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")
```

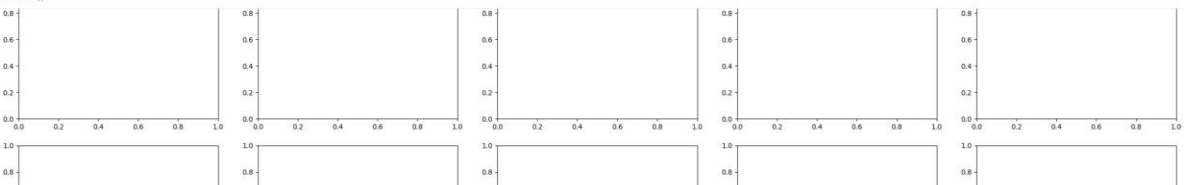
display_cv2_img(img)



```
cap.release()
```

Display Multiple Frames from the Video

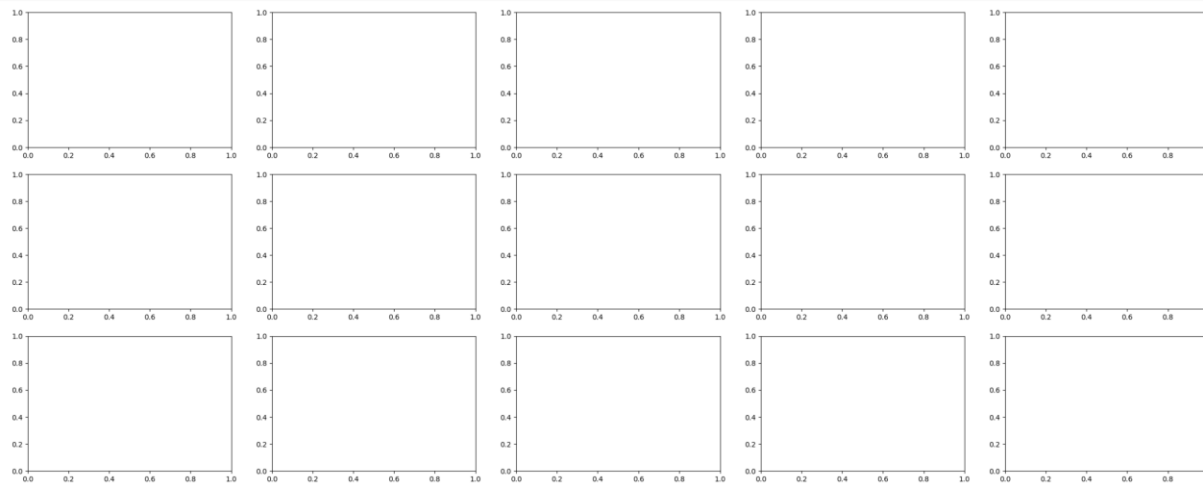
```
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
plt.show()
```





```
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()
plt.show()

# Without Flatten = 2D grid (row, column)
# With flatten = 1D list (single index)
```

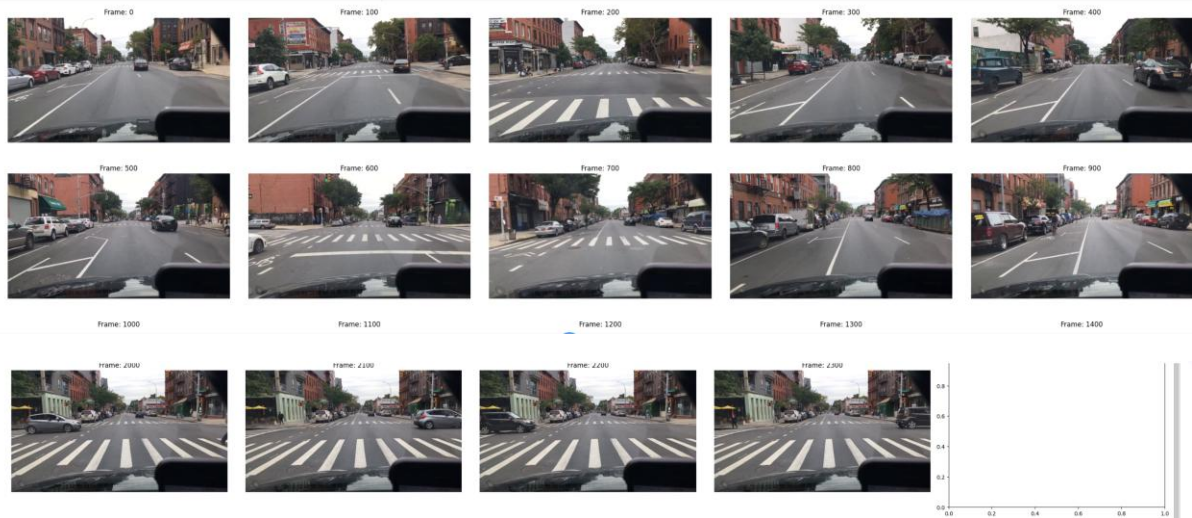


```
import cv2
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'frame: {frame}')
        axs[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()
plt.show()
cap.release()
```



```
labels = pd.read_csv('not_labels.csv', low_memory=False)
#labels.head()
#video_labels = (labels.query('videoName == "026c7465-309f6d33"').reset_index(drop=True).copy())
#video_labels["video_frame"] = (video_labels["frameIndex"] * 11.9).round().astype("int") #Video is at 60Hz but the labellers did only 5 Hz
```

```
#print(video_labels.shape)
```

```
(0, 14)
```

```
print(labels["category"].value_counts())
```

```
category
car          394391
pedestrian   67802
truck        22812
bus          18694
bicycle      3823
other vehicle 3756
rider        3307
motorcycle   2277
other person  782
trailer       376
train         156
Name: count, dtype: int64
```

```
cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame == 1035:
        break
```

```
cap.release()
```

```
) display_cv2_img(img)
```

